

```
// ===== file: config/development.js =====
// Development environment configuration
// Overrides base config for local development
export default {
  port: 3000,
  host: '0.0.0.0',
  nodeEnv: 'development',
  logLevel: 'debug',
  db: {
    host: 'localhost',
    port: 5433,
    name: 'clipsync_dev',
    user: 'clipsync',
    password: 'dev_password_change_me',
    poolMin: 2,
    poolMax: 10,
  },
  redis: {
    host: 'localhost',
    port: 6380,
    password: undefined,
  },
  jwt: {
    secret: 'clipsync-dev-secret',
    expiresIn: '7d',
  },
  cors: {
    origins: '*',
  },
  ws: {
    heartbeatInterval: 30000,
    heartbeatTimeout: 5000,
  },
  rateLimit: {
    api: { windowMs: 15 * 60 * 1000, max: 200 },
    sendCode: { windowMs: 60 * 60 * 1000, max: 10 },
    loginFailed: { windowMs: 15 * 60 * 1000, max: 20 },
  },
  encryption: {
    algorithm: 'aes-256-gcm',
  },
}
```

```
upload: {
  maxImageSize: 20 * 1024 * 1024,
  maxFileSize: 50 * 1024 * 1024,
},
};
// ===== file: config/production.js =====
// Production environment configuration
// All sensitive values should come from environment variables (.env.production)
// This file only provides defaults and production-specific overrides
export default {
  port: parseInt(process.env.PORT || '3000', 10),
  host: process.env.HOST || '0.0.0.0',
  nodeEnv: 'production',
  logLevel: process.env.LOG_LEVEL || 'info',
  db: {
    host: process.env.DB_HOST || 'localhost',
    port: parseInt(process.env.DB_PORT || '5432', 10),
    name: process.env.DB_NAME || 'clipsync',
    user: process.env.DB_USER || 'clipsync_app',
    password: process.env.DB_PASSWORD, // NO default - must be set in .env.production
    poolMin: 5,
    poolMax: 20,
    ssl: process.env.DB_SSL === 'true' ? { rejectUnauthorized: true } : false,
  },
  redis: {
    host: process.env.REDIS_HOST || 'localhost',
    port: parseInt(process.env.REDIS_PORT || '6379', 10),
    password: process.env.REDIS_PASSWORD, // NO default - must be set
  },
  jwt: {
    secret: process.env.JWT_SECRET, // NO default - must be set
    expiresIn: process.env.JWT_EXPIRES_IN || '24h',
  },
  cors: {
    origins: process.env.CORS_ORIGINS || '', // Must whitelist specific origins
  },
  ws: {
    heartbeatInterval: parseInt(process.env.WS_HEARTBEAT_INTERVAL || '30000', 10),
    heartbeatTimeout: parseInt(process.env.WS_HEARTBEAT_TIMEOUT || '5000', 10),
  },
  rateLimit: {
    api: { windowMs: 15 * 60 * 1000, max: 100 },
```

```
sendCode: { windowMs: 60 * 60 * 1000, max: 5 },
loginFailed: { windowMs: 15 * 60 * 1000, max: 5 },
},
encryption: {
algorithm: process.env.ENCRIPTION_ALGORITHM || 'aes-256-gcm',
},
upload: {
maxImageSize: 20 * 1024 * 1024,
maxFileSize: 50 * 1024 * 1024,
},
};
// ===== file: config/test.js =====
// Test environment configuration
// Used for automated testing (vitest, e2e tests)
export default {
port: 3001, // Different port to avoid conflict with dev server
host: '0.0.0.0',
nodeEnv: 'test',
logLevel: 'warn', // Less verbose during tests
db: {
host: 'localhost',
port: 5433,
name: 'clipsync_test',
user: 'clipsync',
password: 'dev_password_change_me',
poolMin: 1,
poolMax: 5,
},
redis: {
host: 'localhost',
port: 6380, // Different Redis port for test isolation
password: 'dev_redis_password',
},
jwt: {
secret: 'clipsync-test-secret',
expiresIn: '1h',
},
cors: {
origins: '*',
},
ws: {
```

```

heartbeatInterval: 5000, // Faster for test speed
heartbeatTimeout: 2000,
},
rateLimit: {
  api: { windowMs: 15 * 60 * 1000, max: 500 }, // More lenient for tests
  sendCode: { windowMs: 60 * 60 * 1000, max: 50 },
  loginFailed: { windowMs: 15 * 60 * 1000, max: 50 },
},
encryption: {
  algorithm: 'aes-256-gcm',
},
upload: {
  maxImageSize: 20 * 1024 * 1024,
  maxFileSize: 50 * 1024 * 1024,
},
};
// ===== file: config.js =====
import 'dotenv/config';
import developmentConfig from './config/development.js';
import testConfig from './config/test.js';
import productionConfig from './config/production.js';
// Select config based on NODE_ENV
const envConfigs = {
  development: developmentConfig,
  test: testConfig,
  production: productionConfig,
};
const nodeEnv = process.env.NODE_ENV || 'development';
const envConfig = envConfigs[nodeEnv] || developmentConfig;
// Deep merge: env vars override config files
function deepMerge(base, override) {
  const result = { ...base };
  for (const key of Object.keys(override)) {
    if (
      typeof base[key] === 'object' && base[key] !== null && !Array.isArray(base[key]) &&
      typeof override[key] === 'object' && override[key] !== null && !Array.isArray(override[key])
    ) {
      result[key] = deepMerge(base[key], override[key]);
    } else if (override[key] !== undefined) {
      result[key] = override[key];
    }
  }
}
return result;

```

```

}
// Environment variable overrides (highest priority)
// NOTE: only include keys when the env var is actually set,
// otherwise leave them out so deepMerge doesn't overwrite with undefined.
const envOverrides = {};
if (process.env.PORT) envOverrides.port = parseInt(process.env.PORT, 10);
if (process.env.HOST) envOverrides.host = process.env.HOST;
if (process.env.LOG_LEVEL) envOverrides.logLevel = process.env.LOG_LEVEL;
// 测试环境强制使用 testConfig 的独立测试库 (clipsync_test) ,
// 避免 .env 中的 DB_NAME=clipsync_dev 等覆盖把整套测试拉到共享 dev 库,
// 造成用例间数据污染与随机失败。其余环境保持原有 env 覆盖行为。
if (nodeEnv !== 'test' && (process.env.DB_HOST || process.env.DB_PORT || process.env.DB_NAME || process.env.DB_USER || process.env.DB_PASSWORD)) {
  envOverrides.db = {};
  if (process.env.DB_HOST) envOverrides.db.host = process.env.DB_HOST;
  if (process.env.DB_PORT) envOverrides.db.port = parseInt(process.env.DB_PORT, 10);
  if (process.env.DB_NAME) envOverrides.db.name = process.env.DB_NAME;
  if (process.env.DB_USER) envOverrides.db.user = process.env.DB_USER;
  if (process.env.DB_PASSWORD) envOverrides.db.password = process.env.DB_PASSWORD;
}
if (process.env.REDIS_HOST || process.env.REDIS_PORT || process.env.REDIS_PASSWORD) {
  envOverrides.redis = {};
  if (process.env.REDIS_HOST) envOverrides.redis.host = process.env.REDIS_HOST;
  if (process.env.REDIS_PORT) envOverrides.redis.port = parseInt(process.env.REDIS_PORT, 10);
  if (process.env.REDIS_PASSWORD) envOverrides.redis.password = process.env.REDIS_PASSWORD;
}
if (process.env.JWT_SECRET) {
  envOverrides.jwt = { ...(envOverrides.jwt || {}), secret: process.env.JWT_SECRET };
}
if (process.env.JWT_EXPIRES_IN) {
  envOverrides.jwt = { ...(envOverrides.jwt || {}), expiresIn: process.env.JWT_EXPIRES_IN };
}
if (process.env.CORS_ORIGINS) {
  envOverrides.cors = { origins: process.env.CORS_ORIGINS.split(',') };
}
if (process.env.WS_HEARTBEAT_INTERVAL || process.env.WS_HEARTBEAT_TIMEOUT) {
  envOverrides.ws = {};
  if (process.env.WS_HEARTBEAT_INTERVAL) envOverrides.ws.heartbeatInterval = parseInt(process.env.WS_HEARTBEAT_INTERVAL, 10);
  if (process.env.WS_HEARTBEAT_TIMEOUT) envOverrides.ws.heartbeatTimeout = parseInt(process.env.WS_HEARTBEAT_TIMEOUT, 10);
}
if (process.env.ENCRYPTION_ALGORITHM) {
  envOverrides.encryption = { algorithm: process.env.ENCRYPTION_ALGORITHM };
}
}

```

```

// Final config: envConfig (base) ← envOverrides (highest priority)
const config = Object.keys(envOverrides).length > 0 ? deepMerge(envConfig, envOverrides) : envConfig;
// Validate critical production settings
if (nodeEnv === 'production') {
  const warnings = [];
  if (!config.jwt.secret || config.jwt.secret === 'clipsync-dev-secret') {
    warnings.push('JWT_SECRET must be set in production (not using dev default)');
  }
  if (!config.db.password) {
    warnings.push('DB_PASSWORD must be set in production');
  }
  if (!config.redis || !config.redis.password) {
    warnings.push('REDIS_PASSWORD should be set in production');
  }
  if (!config.cors || !config.cors.origins || config.cors.origins === '*') {
    warnings.push('CORS_ORIGINS must be explicitly whitelisted in production');
  }
  if (warnings.length > 0) {
    // eslint-disable-next-line no-console
    console.warn('⚠ Production configuration warnings:');
    // eslint-disable-next-line no-console
    warnings.forEach(w => console.warn(' - {w}'));
  }
}
// 默认 JSON / urlencoded body 解析上限 (P1 修复：原 50mb 单请求可吃光内存)
// 文件上传走 multer (diskStorage)，不经由 express.json，故不受此限制。
if (!config.jsonBodyLimit) {
  config.jsonBodyLimit = process.env.JSON_BODY_LIMIT || '10mb';
}
export default config;
// ===== file: crypto/keyExchange.js =====
import crypto from 'crypto';
import { promisify } from 'util';
const generateKeyPairAsync = promisify(crypto.generateKeyPair);
/**
 * ECDH Key Exchange for ClipSync
 *
 * 流程：
 * 1. 客户端生成 ECDH 密钥对
 * 2. 客户端发送公钥到服务器
 * 3. 服务器存储公钥
 * 4. 两个客户端通过对方公钥计算共享密钥
 * 5. 从共享密钥派生加解密密钥

```

```

*/
const CURVE = 'prime256v1'; // NIST P-256
const KEY_LENGTH = 32; // 256 bits
const IV_LENGTH = 12; // 96 bits for AES-GCM
const AUTH_TAG_LENGTH = 16; // 128 bits for AES-GCM
const SALT_LENGTH = 16; // 128 bits for PBKDF2
const PBKDF2_ITERATIONS = 100000;
/**
 * 生成 ECDH 密钥对
 * @returns {Promise<{publicKey: string, privateKey: string}>}
 */
export async function generateKeyPair() {
  const { publicKey, privateKey } = await generateKeyPairAsync('ec', {
    namedCurve: CURVE,
  });
  return {
    publicKey: publicKey.export({ type: 'spki', format: 'pem' }),
    privateKey: privateKey.export({ type: 'pkcs8', format: 'pem' }),
  };
}
/**
 * 从两个公钥计算共享密钥
 * @param {string} privateKey - 本方私钥 (PEM)
 * @param {string} otherPublicKey - 对方公钥 (PEM)
 * @returns {Buffer} 共享密钥
 */
export function computeSharedSecret(privateKey, otherPublicKey) {
  // Use crypto.diffieHellman which correctly handles ECDH key exchange
  return crypto.diffieHellman({
    privateKey: crypto.createPrivateKey(privateKey),
    publicKey: crypto.createPublicKey(otherPublicKey),
  });
}
/**
 * 从共享密钥派生加密密钥 (PBKDF2)
 * @param {Buffer} sharedSecret - 共享密钥
 * @param {Buffer} salt - 盐值
 * @returns {Promise<Buffer>} 派生密钥
 */
export async function deriveKey(sharedSecret, salt) {
  const deriveKeyAsync = promisify(crypto.pbkdf2);
  return deriveKeyAsync(sharedSecret, salt, PBKDF2_ITERATIONS, KEY_LENGTH, 'sha512');
}

```

```
* 生成随机盐值
* @returns {Buffer}
*/
export function generateSalt() {
  return crypto.randomBytes(SALT_LENGTH);
}
/**
* 生成随机 IV
* @returns {Buffer}
*/
export function generateIV() {
  return crypto.randomBytes(IV_LENGTH);
}
/**
* AES-256-GCM 加密
* @param {Buffer|string} plaintext - 明文
* @param {Buffer} key - 256位密钥
* @param {Buffer} iv - 初始化向量
* @returns {{ciphertext: Buffer, authTag: Buffer}}
*/
export function encrypt(plaintext, key, iv) {
  const cipher = crypto.createCipheriv('aes-256-gcm', key, iv, {
    authTagLength: AUTH_TAG_LENGTH,
  });
  const plaintextBuffer = Buffer.isBuffer(plaintext) ? plaintext : Buffer.from(plaintext, 'utf8');
  const ciphertext = Buffer.concat([cipher.update(plaintextBuffer), cipher.final()]);
  const authTag = cipher.getAuthTag();
  return { ciphertext, authTag };
}
/**
* AES-256-GCM 解密
* @param {Buffer} ciphertext - 密文
* @param {Buffer} key - 256位密钥
* @param {Buffer} iv - 初始化向量
* @param {Buffer} authTag - 认证标签
* @returns {Buffer} 明文
*/
export function decrypt(ciphertext, key, iv, authTag) {
  const decipher = crypto.createDecipheriv('aes-256-gcm', key, iv, {
    authTagLength: AUTH_TAG_LENGTH,
  });
  decipher.setAuthTag(authTag);
  const plaintext = Buffer.concat([decipher.update(ciphertext), decipher.final()]);
  return plaintext;
}
```

```

}
/**
 * 完整的加密流程：生成密钥 -> 加密数据
 * @param {string} plaintext - 明文
 * @param {Buffer} key - 加密密钥
 * @returns {{encrypted: string, iv: string, authTag: string, salt: string}}
 */
export function encryptData(plaintext, key) {
  const iv = generateIV();
  const { ciphertext, authTag } = encrypt(plaintext, key, iv);
  return {
    encrypted: ciphertext.toString('base64'),
    iv: iv.toString('base64'),
    authTag: authTag.toString('base64'),
  };
}
/**
 * 完整的解密流程：解密数据
 * @param {string} encrypted - 密文 (base64)
 * @param {Buffer} key - 加密密钥
 * @param {string} iv - 初始化向量 (base64)
 * @param {string} authTag - 认证标签 (base64)
 * @returns {string} 明文
 */
export function decryptData(encrypted, key, iv, authTag) {
  const ciphertext = Buffer.from(encrypted, 'base64');
  const ivBuffer = Buffer.from(iv, 'base64');
  const authTagBuffer = Buffer.from(authTag, 'base64');
  const plaintext = decrypt(ciphertext, key, ivBuffer, authTagBuffer);
  return plaintext.toString('utf8');
}
/**
 * 生成设备密钥对并存储
 * 用于设备间的端到端加密
 */
export class DeviceKeyManager {
  constructor() {
    // 内存存储（生产环境应使用安全存储）
    this.deviceKeys = new Map(); // deviceId -> { publicKey, privateKey }
  }
  /**
   * 为设备生成密钥对
   */
  async generateDeviceKeys(deviceId) {

```

```
const keyPair = await generateKeyPair();
this.deviceKeys.set(deviceId, keyPair);
return keyPair;
}
/**
 * 获取设备公钥
 */
getPublicKey(deviceId) {
const keys = this.deviceKeys.get(deviceId);
return keys ? keys.publicKey : null;
}
/**
 * 获取设备私钥
 */
getPrivateKey(deviceId) {
const keys = this.deviceKeys.get(deviceId);
return keys ? keys.privateKey : null;
}
/**
 * 计算两个设备间的共享密钥
 */
computeSharedKey(deviceId1, deviceId2) {
const keys1 = this.deviceKeys.get(deviceId1);
const keys2 = this.deviceKeys.get(deviceId2);
if (!keys1 || !keys2) {
return null;
}
return computeSharedSecret(keys1.privateKey, keys2.publicKey);
}
/**
 * 加密数据 （从设备1到设备2）
 */
encryptForDevice(plaintext, deviceId1, deviceId2) {
const sharedKey = this.computeSharedKey(deviceId1, deviceId2);
if (!sharedKey) {
return null;
}
const salt = generateSalt();
const iv = generateIV();
const { ciphertext, authTag } = encrypt(plaintext, sharedKey, iv);
return {
encrypted: ciphertext.toString('base64'),
```

```

iv: iv.toString('base64'),
authTag: authTag.toString('base64'),
salt: salt.toString('base64'),
senderDeviceId: deviceId1,
};
}
/**
 * 解密数据（从设备2接收）
 */
decryptFromDevice(encryptedData, deviceId1, deviceId2) {
  const sharedKey = this.computeSharedKey(deviceId1, deviceId2);
  if (!sharedKey) {
    return null;
  }
  const ciphertext = Buffer.from(encryptedData.encrypted, 'base64');
  const iv = Buffer.from(encryptedData.iv, 'base64');
  const authTag = Buffer.from(encryptedData.authTag, 'base64');
  return decrypt(ciphertext, sharedKey, iv, authTag).toString('utf8');
}
}
// 导出单例
export const keyManager = new DeviceKeyManager();
export default {
  generateKeyPair,
  computeSharedSecret,
  deriveKey,
  generateSalt,
  generateIV,
  encrypt,
  decrypt,
  encryptData,
  decryptData,
  keyManager,
};
// ===== file: db/cleanup.js =====
import fs from 'fs/promises';
import path from 'path';
import pool from './pool.js';
import { logger } from '../utils/logger.js';
const CLEANUP_INTERVAL = 60 * 60 * 1000; // 1 hour
export function startCleanupScheduler() {
  // Run cleanup immediately, then every hour

```

```
cleanupExpiredItems();
setInterval(cleanupExpiredItems, CLEANUP_INTERVAL);
}
async function cleanupExpiredItems() {
try {
// Delete expired clipboard items
const result = await pool.query(
`DELETE FROM clipboard_items
WHERE expires_at IS NOT NULL AND expires_at < NOW()
RETURNING id`
);
if (result.rowCount > 0) {
logger.info(`[Cleanup] Deleted ${result.rowCount} expired clipboard items`);
}
// Clean up old verification codes (older than 24 hours)
const vcResult = await pool.query(
`DELETE FROM verification_codes
WHERE created_at < NOW() - INTERVAL '24 hours'
RETURNING id`
);
if (vcResult.rowCount > 0) {
logger.info(`[Cleanup] Deleted ${vcResult.rowCount} old verification codes`);
}
// Clean up old notification history (retention, default 90 days) — P6 修复
const NOTIFICATION_RETENTION_DAYS = parseInt(process.env.NOTIFICATION_RETENTION_DAYS) || 90;
const nhResult = await pool.query(
`DELETE FROM notification_history
WHERE created_at < NOW() - make_interval(days => $1)
RETURNING id`,
[NOTIFICATION_RETENTION_DAYS]
);
if (nhResult.rowCount > 0) {
logger.info(`[Cleanup] Deleted ${nhResult.rowCount} old notification history records (retention ${NOTIFICATION_RETENTION_DAYS}d)`);
}
// Clean up old deletion tombstones (retention, default 30 days)
// 墓碑仅服务于"断线窗口内删除感知", 保留 30 天足够; 防止表无限增长
const TOMBSTONE_RETENTION_DAYS = parseInt(process.env.TOMBSTONE_RETENTION_DAYS) || 30;
const tbResult = await pool.query(
`DELETE FROM clipboard_deletions
WHERE deleted_at < NOW() - make_interval(days => $1)`,
[TOMBSTONE_RETENTION_DAYS]
);
if (tbResult.rowCount > 0) {
logger.info(`[Cleanup] Deleted ${tbResult.rowCount} old deletion tombstones (retention ${TOMBSTONE_RETENTION_DAYS}d)`);
}
```

```

}
// 审计日志先归档后删除（CO-32）：保留天数读 system_configs.audit_log_retention_days
// （管理平台系统参数卡片可调），默认 365；归档失败则跳过删除，宁可超期保留不丢数据。
await archiveAndDeleteExpiredAuditLogs();
} catch (err) {
logger.error('[Cleanup] Error:', { error: err.message });
}
}
// ----- 审计日志归档（CO-32） -----
const AUDIT_ARCHIVE_DIR = path.resolve('logs', 'audit-archive');
const AUDIT_CSV_HEADER =
'id,created_at,user_id,action,resource_type,resource_id,status,ip_address,user_agent,error_message,details';
/** 单元格转 CSV：全字段加引号，内部引号翻倍（details 为 JSON 文本，天然含逗号/引号） */
function csvCell(value) {
if (value === null || value === undefined) return '';
return `${String(value).replace(/"/g, '"')}";`
}
/** Date → 'YYYYMMDD'（本地时区，按归档当天分文件） */
function formatArchiveDate(d) {
const y = d.getFullYear();
const m = String(d.getMonth() + 1).padStart(2, '0');
const day = String(d.getDate()).padStart(2, '0');
return `${y}${m}${day}`;
}
/** 读 system_configs.audit_log_retention_days（JSONB 数字或字符串；非法值回退 365） */
async function readAuditRetentionDays() {
const { rows } = await pool.query(
'SELECT config_value FROM system_configs WHERE config_key = \'audit_log_retention_days\'
');
const days = parseInt(rows[0]?.config_value, 10);
return Number.isFinite(days) && days > 0 ? days : 365;
}
/**
* 归档并删除超期 audit_logs 行：
* 1. 读保留天数（system_configs.audit_log_retention_days，默认 365）
* 2. 捞出超期行 → 追加导出 CSV（logs/audit-archive/audit-YYYYMMDD.csv，logs 目录容器内已挂载）
* 3. 按 id 精确删除已归档行（CSV 写失败时抛错 → 上层跳过删除，不丢审计数据）
*/
async function archiveAndDeleteExpiredAuditLogs() {
let retentionDays;
try {
retentionDays = await readAuditRetentionDays();
} catch (err) {

```

```
// system_configs 不可达：跳过本轮（cleanup 整体已有 error 日志，这里补充语义）
logger.warn('[Cleanup] audit retention read failed, skip archive this round', { error: err.message });
return;
}

const { rows } = await pool.query(
  `SELECT id, created_at, user_id, action, resource_type, resource_id, status,
ip_address::text AS ip_address, user_agent, error_message, details
FROM audit_logs
WHERE created_at < NOW() - make_interval(days => $1)
ORDER BY created_at ASC`,
  [retentionDays]
);
if (rows.length === 0) return;
const csvFile = path.join(AUDIT_ARCHIVE_DIR, `audit-${formatArchiveDate(new Date())}.csv`);
const lines = rows.map((row) =>
[
  row.id,
  row.created_at instanceof Date ? row.created_at.toISOString() : row.created_at,
  row.user_id,
  row.action,
  row.resource_type,
  row.resource_id,
  row.status,
  row.ip_address,
  row.user_agent,
  row.error_message,
  row.details === null ? '' : JSON.stringify(row.details),
]
).map(csvCell)
.join(',')
);
try {
  await fs.mkdir(AUDIT_ARCHIVE_DIR, { recursive: true });
  let content = lines.join("\n") + "\n";
  try {
    const existing = await fs.stat(csvFile);
    if (existing.size === 0) content = AUDIT_CSV_HEADER + "\n" + content;
  } catch {}
  // 文件不存在：本文件首个数据块，写表头
  content = AUDIT_CSV_HEADER + "\n" + content;
}
await fs.appendFile(csvFile, content);
} catch (err) {
  logger.error('[Cleanup] audit archive CSV write failed, skip delete', {
    file: csvFile,
    error: err.message,
  });
};
```

```

return;
}
const ids = rows.map((r) => r.id);
const delResult = await pool.query('DELETE FROM audit_logs WHERE id = ANY($1)', [ids]);
logger.info(
  `[Cleanup] Archived ${rows.length} expired audit logs to ${csvFile} and deleted ${delResult.rowCount} rows (retention ${retentionDays}d)`
);
}
// ===== file: db/migrate-manager.js =====
import pool from './pool.js';
import { logger } from '../utils/logger.js';
/**
 * ClipSync Database Migration Manager
 * Handles schema version upgrades and data migrations
 */
const SCHEMA_VERSIONS = [
  {
    version: 1,
    description: 'Initial schema - users, devices, clipboard_items, device_sync_state, verification_codes',
    migrations: [], // Initial schema is created by migrate.js
  },
  {
    version: 2,
    description: 'Add updated_at column to clipboard_items, add file_versions table',
    migrations: [
      'ALTER TABLE clipboard_items ADD COLUMN IF NOT EXISTS updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()',
      // file_versions already in migrate.js, but this handles existing deployments
      'CREATE TABLE IF NOT EXISTS file_versions (
        id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
        clipboard_item_id UUID NOT NULL REFERENCES clipboard_items(id) ON DELETE CASCADE,
        user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
        version_number INTEGER NOT NULL DEFAULT 1,
        content_encrypted TEXT NOT NULL,
        content_preview TEXT DEFAULT "",
        content_size INTEGER DEFAULT 0,
        metadata JSONB DEFAULT '{}',
        source_device_id UUID REFERENCES devices(id) ON DELETE SET NULL,
        change_description TEXT DEFAULT "",
        created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
        UNIQUE(clipboard_item_id, version_number)
      )',
    ],
  },
  {
    version: 3,
    description: 'Add browser device_type support, add search_vector for full-text search',
  },

```

```

migrations: [
// Update devices CHECK constraint to include 'browser'
`ALTER TABLE devices DROP CONSTRAINT IF EXISTS devices_device_type_check`,
`ALTER TABLE devices ADD CONSTRAINT devices_device_type_check CHECK (device_type IN ('desktop', 'mobile', 'tablet', 'browser'))`,
`ALTER TABLE devices DROP CONSTRAINT IF EXISTS devices_platform_check`,
`ALTER TABLE devices ADD CONSTRAINT devices_platform_check CHECK (platform IN ('windows', 'macos', 'linux', 'ios', 'android', 'browser'))`,
// Add search_vector tsvector column
`DO $$ BEGIN
IF NOT EXISTS (SELECT 1 FROM information_schema.columns WHERE table_name = 'clipboard_items' AND column_name = 'search_vector') THEN
ALTER TABLE clipboard_items ADD COLUMN search_vector tsvector;
END IF;
END $$`,
// Create trigger for auto-updating search_vector
`CREATE OR REPLACE FUNCTION clipsync_update_search_vector() RETURNS trigger AS $$ BEGIN
NEW.search_vector := setweight(to_tsvector('simple', coalesce(NEW.content_type, '')), 'A') || setweight(to_tsvector('simple', coalesce(NEW.content_preview, '')),
'B') || setweight(to_tsvector('simple', coalesce(NEW.ocr_text, '')), 'C');
RETURN NEW;
END $$ LANGUAGE plpgsql`,
`DROP TRIGGER IF EXISTS clipsync_search_vector_update ON clipboard_items`,
`CREATE TRIGGER clipsync_search_vector_update BEFORE INSERT OR UPDATE OF content_preview, content_type, ocr_text ON clipboard_items FOR
EACH ROW EXECUTE FUNCTION clipsync_update_search_vector()`,
// GIN index for search_vector
`CREATE INDEX IF NOT EXISTS idx_clipboard_search ON clipboard_items USING GIN(search_vector)`,
],
},
{
version: 4,
description: 'Add content_diff column for incremental sync',
migrations: [
`DO $$ BEGIN
IF NOT EXISTS (SELECT 1 FROM information_schema.columns WHERE table_name = 'clipboard_items' AND column_name = 'content_diff') THEN
ALTER TABLE clipboard_items ADD COLUMN content_diff TEXT DEFAULT NULL;
END IF;
END $$`,
],
},
{
version: 5,
description: 'Add payment & subscription tables (subscription_plans, user_subscriptions, payment_orders, invoices) + extend users table',
migrations: [
// Extend users table
`DO $$ BEGIN
IF NOT EXISTS (SELECT 1 FROM information_schema.columns WHERE table_name = 'users' AND column_name = 'subscription_status') THEN
ALTER TABLE users ADD COLUMN subscription_status VARCHAR(20) DEFAULT 'free';
END IF;
END $$`,
`DO $$ BEGIN
IF NOT EXISTS (SELECT 1 FROM information_schema.columns WHERE table_name = 'users' AND column_name = 'current_subscription_id') THEN
ALTER TABLE users ADD COLUMN current_subscription_id UUID;
END IF;
END $$`,

```

```

// Create subscription_plans table
`CREATE TABLE IF NOT EXISTS subscription_plans (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
name VARCHAR(50) NOT NULL,
price DECIMAL(10,2) NOT NULL DEFAULT 0,
currency VARCHAR(10) DEFAULT 'CNY',
billing_cycle VARCHAR(20) DEFAULT 'monthly',
max_devices INTEGER DEFAULT 2,
max_clipboard_items INTEGER DEFAULT 50,
max_file_size_mb INTEGER DEFAULT 1,
max_storage_mb INTEGER DEFAULT 100,
features JSONB DEFAULT '{}',
is_active BOOLEAN DEFAULT true,
created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

// Create user_subscriptions table
`CREATE TABLE IF NOT EXISTS user_subscriptions (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
plan_id UUID NOT NULL REFERENCES subscription_plans(id),
status VARCHAR(20) DEFAULT 'active',
current_period_start TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
current_period_end TIMESTAMP WITH TIME ZONE,
cancel_at_period_end BOOLEAN DEFAULT false,
trial_end TIMESTAMP WITH TIME ZONE,
created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

// Create payment_orders table
`CREATE TABLE IF NOT EXISTS payment_orders (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
subscription_id UUID REFERENCES user_subscriptions(id) ON DELETE SET NULL,
order_no VARCHAR(100) UNIQUE NOT NULL,
amount DECIMAL(10,2) NOT NULL,
currency VARCHAR(10) DEFAULT 'CNY',
payment_method VARCHAR(50),
status VARCHAR(20) DEFAULT 'pending',
paid_at TIMESTAMP WITH TIME ZONE,
transaction_id VARCHAR(200),
metadata JSONB DEFAULT '{}',
created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

// Create invoices table
`CREATE TABLE IF NOT EXISTS invoices (

```

```

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
order_id UUID REFERENCES payment_orders(id) ON DELETE SET NULL,
invoice_no VARCHAR(100) UNIQUE NOT NULL,
amount DECIMAL(10,2) NOT NULL,
tax DECIMAL(10,2) DEFAULT 0,
status VARCHAR(20) DEFAULT 'draft',
invoice_url TEXT,
created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
),
// Add foreign key for users.current_subscription_id
`DO $$ BEGIN
IF NOT EXISTS (SELECT 1 FROM information_schema.table_constraints WHERE constraint_name = 'users_current_subscription_id_fkey') THEN
ALTER TABLE users ADD CONSTRAINT users_current_subscription_id_fkey
FOREIGN KEY (current_subscription_id) REFERENCES user_subscriptions(id) ON DELETE SET NULL;
END IF;
END $$`,
// Insert default subscription plans
`INSERT INTO subscription_plans (name, display_name, description, price_monthly, price_yearly, max_devices, max_clipboard_items, max_file_size_mb,
max_storage_mb, features)
SELECT 'Free', '免费版', '基础功能', 0, 0, 2, 50, 1, 100, '{"ai_classify": true, "offline_queue": true, "e2e_encryption": true, "push_notification": false, "full_text_search":
false, "version_history_days": 3}':jsonb
WHERE NOT EXISTS (SELECT 1 FROM subscription_plans WHERE name = 'Free')`,
`INSERT INTO subscription_plans (name, display_name, description, price_monthly, price_yearly, max_devices, max_clipboard_items, max_file_size_mb,
max_storage_mb, features)
SELECT 'Pro', '专业版', '高级功能', 9.9, 99, 5, 999999, 20, 5120, '{"ai_classify": true, "offline_queue": true, "e2e_encryption": true, "push_notification": true,
"full_text_search": true, "version_history_days": 30}':jsonb
WHERE NOT EXISTS (SELECT 1 FROM subscription_plans WHERE name = 'Pro')`,
`INSERT INTO subscription_plans (name, display_name, description, price_monthly, price_yearly, max_devices, max_clipboard_items, max_file_size_mb,
max_storage_mb, features)
SELECT 'Enterprise', '企业版', '企业级功能', 29.9, 299, 999999, 999999, 100, 51200, '{"ai_classify": true, "offline_queue": true, "e2e_encryption": true,
"push_notification": true, "full_text_search": true, "version_history_days": -1}':jsonb
WHERE NOT EXISTS (SELECT 1 FROM subscription_plans WHERE name = 'Enterprise')`,
// Create indexes
`CREATE INDEX IF NOT EXISTS idx_user_subscriptions_user_id ON user_subscriptions(user_id)`,
`CREATE INDEX IF NOT EXISTS idx_user_subscriptions_status ON user_subscriptions(status)`,
`CREATE INDEX IF NOT EXISTS idx_payment_orders_user_id ON payment_orders(user_id)`,
`CREATE INDEX IF NOT EXISTS idx_payment_orders_order_no ON payment_orders(order_no)`,
`CREATE INDEX IF NOT EXISTS idx_invoices_user_id ON invoices(user_id)`,
`CREATE INDEX IF NOT EXISTS idx_invoices_invoice_no ON invoices(invoice_no)`
],
},
{
version: 6,
description: 'Add encryption fields for sensitive data (phone, email, payment tokens)',
migrations: [
// Add encrypted fields to users table
`DO $$ BEGIN
IF NOT EXISTS (SELECT 1 FROM information_schema.columns WHERE table_name = 'users' AND column_name = 'phone_encrypted') THEN
ALTER TABLE users ADD COLUMN phone_encrypted TEXT DEFAULT NULL;

```

```

END IF;
IF NOT EXISTS (SELECT 1 FROM information_schema.columns WHERE table_name = 'users' AND column_name = 'email_encrypted') THEN
ALTER TABLE users ADD COLUMN email_encrypted TEXT DEFAULT NULL;
END IF;
END $$;
// Add encrypted fields to payment_orders table
`DO $$ BEGIN
IF NOT EXISTS (SELECT 1 FROM information_schema.columns WHERE table_name = 'payment_orders' AND column_name = 'payment_token_encrypted')
THEN
ALTER TABLE payment_orders ADD COLUMN payment_token_encrypted TEXT DEFAULT NULL;
END IF;
END $$;
// Add encrypted fields to user_subscriptions table
`DO $$ BEGIN
IF NOT EXISTS (SELECT 1 FROM information_schema.columns WHERE table_name = 'user_subscriptions' AND column_name =
'subscription_token_encrypted') THEN
ALTER TABLE user_subscriptions ADD COLUMN subscription_token_encrypted TEXT DEFAULT NULL;
END IF;
END $$;
// Create encryption_keys table for key management
`CREATE TABLE IF NOT EXISTS encryption_keys (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
key_name VARCHAR(100) NOT NULL UNIQUE,
key_value TEXT NOT NULL,
iv TEXT NOT NULL,
algorithm VARCHAR(50) DEFAULT 'AES-256-GCM',
is_active BOOLEAN DEFAULT true,
created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
),
// Create index for encryption_keys
`CREATE INDEX IF NOT EXISTS idx_encryption_keys_active ON encryption_keys(is_active)`,
],
},
{
version: 7,
description: 'Add user_sessions table for session management',
migrations: [
// Create user_sessions table
`CREATE TABLE IF NOT EXISTS user_sessions (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
device_id UUID REFERENCES devices(id) ON DELETE SET NULL,
session_token_hash TEXT NOT NULL,
ip_address INET,
user_agent TEXT,
created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
last_active_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),

```

```

is_active BOOLEAN DEFAULT true
),
// Create index for user_sessions
`CREATE INDEX IF NOT EXISTS idx_user_sessions_user_id ON user_sessions(user_id)`,
`CREATE INDEX IF NOT EXISTS idx_user_sessions_is_active ON user_sessions(is_active)`,
],
},
{
version: 8,
description: 'Add ocr_text to clipboard_items and include it in search_vector (OCR preprocessing for images, #224)',
migrations: [
// 1) 新增可搜索的文字层列（图片 OCR 提取结果）
`DO $$ BEGIN
IF NOT EXISTS (SELECT 1 FROM information_schema.columns WHERE table_name = 'clipboard_items' AND column_name = 'ocr_text') THEN
ALTER TABLE clipboard_items ADD COLUMN ocr_text TEXT DEFAULT NULL;
END IF;
END $$`,
// 2) 触发器函数纳入 ocr_text（权重 C），CREATE OR REPLACE 幂等
`CREATE OR REPLACE FUNCTION clipsync_update_search_vector() RETURNS trigger AS $$
BEGIN
NEW.search_vector :=
setweight(to_tsvector('simple', coalesce(NEW.content_type, '')), 'A') ||
setweight(to_tsvector('simple', coalesce(NEW.content_preview, '')), 'B') ||
setweight(to_tsvector('simple', coalesce(NEW.ocr_text, '')), 'C');
RETURN NEW;
END
$$ LANGUAGE plpgsql`,
// 3) 重建触发器，把 ocr_text 纳入触发列（否则仅更新 ocr_text 不会刷新 search_vector）
`DROP TRIGGER IF EXISTS clipsync_search_vector_update ON clipboard_items`,
`CREATE TRIGGER clipsync_search_vector_update BEFORE INSERT OR UPDATE OF content_preview, content_type, ocr_text ON clipboard_items FOR
EACH ROW EXECUTE FUNCTION clipsync_update_search_vector()`,
// 4) 回填：把已有（当前为空）ocr_text 纳入 search_vector；列为 NULL 时等价于补空，无副作用
`UPDATE clipboard_items
SET search_vector = setweight(to_tsvector('simple', coalesce(content_type, '')), 'A') ||
setweight(to_tsvector('simple', coalesce(content_preview, '')), 'B') ||
setweight(to_tsvector('simple', coalesce(ocr_text, '')), 'C')
WHERE ocr_text IS NOT NULL`,
],
},
{
version: 9,
description: 'Persist AI conversation token usage (prompt/completion/cache/thinking)',
migrations: [
`DO $$ BEGIN
IF NOT EXISTS (SELECT 1 FROM information_schema.columns WHERE table_name = 'ai_conversations' AND column_name = 'prompt_tokens') THEN
ALTER TABLE ai_conversations ADD COLUMN prompt_tokens INTEGER DEFAULT 0;
END IF;

```

```

IF NOT EXISTS (SELECT 1 FROM information_schema.columns WHERE table_name = 'ai_conversations' AND column_name = 'completion_tokens') THEN
ALTER TABLE ai_conversations ADD COLUMN completion_tokens INTEGER DEFAULT 0;
END IF;
IF NOT EXISTS (SELECT 1 FROM information_schema.columns WHERE table_name = 'ai_conversations' AND column_name = 'total_tokens') THEN
ALTER TABLE ai_conversations ADD COLUMN total_tokens INTEGER DEFAULT 0;
END IF;
IF NOT EXISTS (SELECT 1 FROM information_schema.columns WHERE table_name = 'ai_conversations' AND column_name = 'cache_read_tokens') THEN
ALTER TABLE ai_conversations ADD COLUMN cache_read_tokens INTEGER DEFAULT 0;
END IF;
IF NOT EXISTS (SELECT 1 FROM information_schema.columns WHERE table_name = 'ai_conversations' AND column_name = 'cache_write_tokens') THEN
ALTER TABLE ai_conversations ADD COLUMN cache_write_tokens INTEGER DEFAULT 0;
END IF;
IF NOT EXISTS (SELECT 1 FROM information_schema.columns WHERE table_name = 'ai_conversations' AND column_name = 'cache_hit_rate') THEN
ALTER TABLE ai_conversations ADD COLUMN cache_hit_rate NUMERIC(5,2) DEFAULT 0;
END IF;
IF NOT EXISTS (SELECT 1 FROM information_schema.columns WHERE table_name = 'ai_conversations' AND column_name = 'thinking_tokens') THEN
ALTER TABLE ai_conversations ADD COLUMN thinking_tokens INTEGER DEFAULT 0;
END IF;
IF NOT EXISTS (SELECT 1 FROM information_schema.columns WHERE table_name = 'ai_conversations' AND column_name = 'reply_tokens') THEN
ALTER TABLE ai_conversations ADD COLUMN reply_tokens INTEGER DEFAULT 0;
END IF;
IF NOT EXISTS (SELECT 1 FROM information_schema.columns WHERE table_name = 'ai_conversations' AND column_name = 'context_window') THEN
ALTER TABLE ai_conversations ADD COLUMN context_window INTEGER DEFAULT 0;
END IF;
END $$,
],
},
{
version: 10,
description: 'Image content hash for cross-copy dedup and AI duplicate detection (#225)',
migrations: [
`ALTER TABLE clipboard_items ADD COLUMN IF NOT EXISTS image_hash VARCHAR(64);`,
`CREATE INDEX IF NOT EXISTS idx_clipboard_user_image_hash ON clipboard_items (user_id, image_hash) WHERE image_hash IS NOT NULL;`,
],
},
];
// Create schema_versions tracking table if not exists
async function ensureVersionTable() {
await pool.query(`
CREATE TABLE IF NOT EXISTS schema_versions (
version INTEGER PRIMARY KEY,
description TEXT NOT NULL,
applied_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
)
`);
}
// Get current schema version
async function getCurrentVersion() {

```

```
const result = await pool.query('SELECT MAX(version) as version FROM schema_versions');
return result.rows[0]?.version || 0;
}

// Mark a version as applied
async function markVersionApplied(version, description) {
  await pool.query(
    'INSERT INTO schema_versions (version, description) VALUES ($1, $2) ON CONFLICT (version) DO NOTHING',
    [version, description]
  );
}

// Run migrations from current version to target version
async function runMigrations(targetVersion = null) {
  await ensureVersionTable();
  const currentVersion = await getCurrentVersion();
  const maxVersion = SCHEMA_VERSIONS[SCHEMA_VERSIONS.length - 1].version;
  const target = targetVersion || maxVersion;
  logger.info('Migration manager starting', { currentVersion, targetVersion: target });
  if (currentVersion >= target) {
    logger.info('No migrations needed', { currentVersion, targetVersion: target });
    return { applied: 0, currentVersion };
  }
  let applied = 0;
  const client = await pool.connect();
  try {
    for (const schemaVersion of SCHEMA_VERSIONS) {
      if (schemaVersion.version <= currentVersion) continue;
      if (schemaVersion.version > target) break;
      logger.info('Applying schema version ${schemaVersion.version}: ${schemaVersion.description}');
      for (const migration of schemaVersion.migrations) {
        try {
          await client.query(migration);
          logger.info(' Migration applied: ${migration.substring(0, 80)}...');
        } catch (err) {
          logger.error(' Migration failed: ${err.message}', { migration: migration.substring(0, 100) });
          // Rollback this version's migrations
          throw new Error(`Schema version ${schemaVersion.version} migration failed: ${err.message}`);
        }
      }
      await markVersionApplied(schemaVersion.version, schemaVersion.description);
      applied++;
      logger.info('Schema version ${schemaVersion.version} applied successfully');
    }
  }
}
```

```

}
} catch (err) {
  logger.error('Migration failed, rolling back', { error: err.message });
  // Re-apply previous version marker
  throw err;
} finally {
  client.release();
}
logger.info('All migrations completed', { applied, newVersion: await getCurrentVersion() });
return { applied, currentVersion: await getCurrentVersion() };
}
// Backfill search_vector for existing rows
async function backfillSearchVector() {
  logger.info('Backfilling search_vector for existing clipboard items...');
  const result = await pool.query(
    `UPDATE clipboard_items
    SET search_vector = setweight(to_tsvector('simple', coalesce(content_type, '')), 'A') ||
    setweight(to_tsvector('simple', coalesce(content_preview, '')), 'B') ||
    setweight(to_tsvector('simple', coalesce(ocr_text, '')), 'C')
    WHERE search_vector IS NULL OR ocr_text IS NOT NULL
  `);
  logger.info('Backfill completed: ${result.rowCount} rows updated');
  return result.rowCount;
}
// Export migration manager
export { runMigrations, getCurrentVersion, backfillSearchVector, SCHEMA_VERSIONS };
// Run directly from command line
if (process.argv[1] && process.argv[1].includes('migrate-manager')) {
  const command = process.argv[2] || 'migrate';
  switch (command) {
    case 'migrate':
      runMigrations()
        .then(result => {
          logger.info('Migrations applied: ${result.applied}, current version: ${result.currentVersion}');
          // Backfill search_vector if needed
          return backfillSearchVector();
        })
        .then(count => {
          logger.info('Search vector backfill: ${count} rows');
          process.exit(0);
        })
        .catch(err => {
          logger.error('Migration failed:', { error: err.message });
          process.exit(1);
        });
  }
}

```

```

break;
case 'status':
ensureVersionTable()
.then(() => getCurrentVersion())
.then(version => {
logger.info('Current schema version: ${version}');
logger.info('Available versions: ${SCHEMA_VERSIONS.map(v => `${v.version}: ${v.description}`).join("\n")});
process.exit(0);
})
.catch(err => {
logger.error('Status check failed:', { error: err.message });
process.exit(1);
});
break;
case 'backfill':
backfillSearchVector()
.then(count => {
logger.info('Backfill completed: ${count} rows updated');
process.exit(0);
})
.catch(err => {
logger.error('Backfill failed:', { error: err.message });
process.exit(1);
});
break;
default:
logger.info('Usage: node migrate-manager.js [migrate|status|backfill]');
process.exit(0);
}
}
// ===== file: db/migrate.js =====
import pool from './pool.js';
import { logger } from '../utils/logger.js';
import fs from 'fs';
import path from 'path';
const migrations = [
// 1. Users table (complete schema with all fields required by auth/subscription routes)
`CREATE TABLE IF NOT EXISTS users (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
phone VARCHAR(20) UNIQUE NOT NULL,
email VARCHAR(255),
nickname VARCHAR(100) DEFAULT "",
avatar_url TEXT DEFAULT "",
password_hash VARCHAR(255),
phone_encrypted TEXT,

```

```

email_encrypted TEXT,
phone_hash VARCHAR(128),
email_hash VARCHAR(128),
tos_accepted_at TIMESTAMPTZ,
privacy_accepted_at TIMESTAMPTZ,
marketing_consent BOOLEAN DEFAULT FALSE,
birth_date DATE,
age_verified BOOLEAN DEFAULT FALSE,
is_active BOOLEAN DEFAULT TRUE,
deactivated_at TIMESTAMPTZ,
deactivation_reason TEXT,
analytics_consent BOOLEAN,
functional_consent BOOLEAN,
consent_updated_at TIMESTAMPTZ,
subscription_status VARCHAR(20) DEFAULT 'free',
current_subscription_id UUID,
is_admin BOOLEAN DEFAULT false,
created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
// 2. Devices table
`CREATE TABLE IF NOT EXISTS devices (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
device_name VARCHAR(100) NOT NULL,
device_type VARCHAR(20) NOT NULL CHECK (device_type IN ('desktop', 'mobile', 'tablet', 'browser')),
platform VARCHAR(20) NOT NULL CHECK (platform IN ('windows', 'macos', 'linux', 'ios', 'android', 'browser')),
platform_version VARCHAR(50) DEFAULT '',
app_version VARCHAR(20) DEFAULT '0.1.0',
public_key TEXT,
is_online BOOLEAN DEFAULT FALSE,
last_seen_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
UNIQUE(user_id, device_name)
);
// 3. Clipboard items table
`CREATE TABLE IF NOT EXISTS clipboard_items (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
source_device_id UUID NOT NULL REFERENCES devices(id) ON DELETE CASCADE,
content_type VARCHAR(20) NOT NULL CHECK (content_type IN ('text', 'image', 'file', 'link', 'code')),
content_encrypted TEXT NOT NULL,
content_preview TEXT DEFAULT '',
content_size INTEGER DEFAULT 0,
metadata JSONB DEFAULT '{}',
is_favorite BOOLEAN DEFAULT FALSE,
expires_at TIMESTAMP WITH TIME ZONE,
created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),

```

```

updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
)',
// 4. Device sync state table
`CREATE TABLE IF NOT EXISTS device_sync_state (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
device_id UUID UNIQUE NOT NULL REFERENCES devices(id) ON DELETE CASCADE,
last_synced_item_id UUID,
last_sync_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
)',
// 5. Verification codes table
`CREATE TABLE IF NOT EXISTS verification_codes (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
phone VARCHAR(20) NOT NULL,
code VARCHAR(10) NOT NULL,
expires_at TIMESTAMP WITH TIME ZONE NOT NULL,
used BOOLEAN DEFAULT FALSE,
created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
)',
// 6. User sessions table (auth login requires)
`CREATE TABLE IF NOT EXISTS user_sessions (
id UUID PRIMARY KEY,
user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
device_name VARCHAR(100) DEFAULT 'Unknown Device',
device_type VARCHAR(20) DEFAULT 'browser',
platform VARCHAR(20) DEFAULT 'unknown',
ip_address VARCHAR(45),
user_agent TEXT,
is_active BOOLEAN DEFAULT TRUE,
created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
revoked_at TIMESTAMP WITH TIME ZONE
)',
// 7. Subscription plans table (subscriptionCheck middleware requires)
`CREATE TABLE IF NOT EXISTS subscription_plans (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
name VARCHAR(50) UNIQUE NOT NULL,
display_name VARCHAR(100) NOT NULL,
description TEXT,
price_monthly DECIMAL(10,2),
price_yearly DECIMAL(10,2),
max_devices INTEGER DEFAULT 2,
max_clipboard_items INTEGER DEFAULT 50,
max_file_size_mb INTEGER DEFAULT 1,
max_storage_mb INTEGER DEFAULT 100,
features JSONB DEFAULT '{}',
is_active BOOLEAN DEFAULT true,
created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()

```

```

)',
// 8. User subscriptions table (subscriptionCheck middleware requires)
`CREATE TABLE IF NOT EXISTS user_subscriptions (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
plan_id UUID NOT NULL REFERENCES subscription_plans(id),
status VARCHAR(20) DEFAULT 'active' CHECK (status IN ('active','canceled','past_due','expired')),
billing_cycle VARCHAR(10) DEFAULT 'monthly' CHECK (billing_cycle IN ('monthly','yearly')),
current_period_start TIMESTAMP WITH TIME ZONE NOT NULL,
current_period_end TIMESTAMP WITH TIME ZONE NOT NULL,
start_date TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
end_date TIMESTAMP WITH TIME ZONE,
stripe_subscription_id VARCHAR(255),
alipay_agreement_id VARCHAR(255),
wechat_pay_prepay_id VARCHAR(255),
created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
)',
// 6. File versions table
`CREATE TABLE IF NOT EXISTS file_versions (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
clipboard_item_id UUID NOT NULL REFERENCES clipboard_items(id) ON DELETE CASCADE,
user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
version_number INTEGER NOT NULL DEFAULT 1,
content_encrypted TEXT NOT NULL,
content_preview TEXT DEFAULT '',
content_size INTEGER DEFAULT 0,
metadata JSONB DEFAULT '{}',
source_device_id UUID REFERENCES devices(id) ON DELETE SET NULL,
change_description TEXT DEFAULT '',
created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
UNIQUE(clipboard_item_id, version_number)
)',
// 7. Indexes
`CREATE INDEX IF NOT EXISTS idx_devices_user_id ON devices(user_id)`,
`CREATE INDEX IF NOT EXISTS idx_devices_online ON devices(is_online) WHERE is_online = TRUE`,
`CREATE INDEX IF NOT EXISTS idx_clipboard_items_user_id ON clipboard_items(user_id)`,
`CREATE INDEX IF NOT EXISTS idx_clipboard_items_created_at ON clipboard_items(created_at DESC)`,
`CREATE INDEX IF NOT EXISTS idx_clipboard_items_content_type ON clipboard_items(content_type)`,
`CREATE INDEX IF NOT EXISTS idx_clipboard_items_favorites ON clipboard_items(is_favorite) WHERE is_favorite = TRUE`,
`CREATE INDEX IF NOT EXISTS idx_verification_codes_phone ON verification_codes(phone, used) WHERE used = FALSE`,
`CREATE INDEX IF NOT EXISTS idx_file_versions_item ON file_versions(clipboard_item_id, version_number)`,
`CREATE INDEX IF NOT EXISTS idx_file_versions_user ON file_versions(user_id)`,
`CREATE INDEX IF NOT EXISTS idx_user_sessions_user_id ON user_sessions(user_id)`,
`CREATE INDEX IF NOT EXISTS idx_user_sessions_active ON user_sessions(user_id, is_active) WHERE is_active = TRUE`,
// User lookup indexes (O(1) dedup + login)
`CREATE INDEX IF NOT EXISTS idx_users_phone_hash ON users(phone_hash) WHERE phone_hash IS NOT NULL`,

```

ClipSync 剪贴板同步软件 V1.0.0 第 28 页

```
`CREATE INDEX IF NOT EXISTS idx_users_email_hash ON users(email_hash) WHERE email_hash IS NOT NULL`,
`CREATE UNIQUE INDEX IF NOT EXISTS idx_users_email_unique ON users(email) WHERE email IS NOT NULL AND email != '',
// Identity merge: track merged-into canonical user id
`ALTER TABLE users ADD COLUMN IF NOT EXISTS merged_into UUID REFERENCES users(id) ON DELETE SET NULL`,
];
// Post-migration: add tsvector column and trigger if not exists
const postMigrations = [
// Add search_vector column to clipboard_items (safe for existing tables)
`DO $$
BEGIN
IF NOT EXISTS (SELECT 1 FROM information_schema.columns WHERE table_name = 'clipboard_items' AND column_name = 'search_vector') THEN
ALTER TABLE clipboard_items ADD COLUMN search_vector tsvector;
END IF;
END $$`,
// Create trigger function to auto-update search_vector from content_preview
`CREATE OR REPLACE FUNCTION clipsync_update_search_vector() RETURNS trigger AS $$
BEGIN
NEW.search_vector :=
setweight(to_tsvector('simple', coalesce(NEW.content_type, '')), 'A') ||
setweight(to_tsvector('simple', coalesce(NEW.content_preview, '')), 'B') ||
setweight(to_tsvector('simple', coalesce(NEW.ocr_text, '')), 'C');
RETURN NEW;
END
$$ LANGUAGE plpgsql`,
// Create GIN index (must be after column and trigger are created)
`CREATE INDEX IF NOT EXISTS idx_clipboard_search ON clipboard_items USING GIN(search_vector)`,
// Seed subscription plans if empty
`INSERT INTO subscription_plans (name, display_name, description, price_monthly, price_yearly, max_devices, max_clipboard_items, max_file_size_mb,
max_storage_mb, features)
VALUES
('Free', '免费版', '基础剪贴板同步功能', 0, 0, 2, 50, 1, 100,
'{"ai_classify":true,"offline_queue":true,"e2e_encryption":true,"push_notification":false,"full_text_search":false,"version_history_days":3}'),
('Pro', '专业版', '完整功能解锁', 9.9, 99, 10, 500, 10, 1024,
'{"ai_classify":true,"offline_queue":true,"e2e_encryption":true,"push_notification":true,"full_text_search":true,"version_history_days":30}')
ON CONFLICT (name) DO NOTHING`,
// Archive feature: add archived flag (idempotent, additive column)
`ALTER TABLE clipboard_items ADD COLUMN IF NOT EXISTS archived BOOLEAN NOT NULL DEFAULT FALSE`,
`CREATE INDEX IF NOT EXISTS idx_clipboard_items_archived ON clipboard_items(archived, created_at DESC)`,
];
async function migrate() {
logger.info('Running database migrations...');
const client = await pool.connect();
try {
// 创建 schema_migrations 表（如果不存在）
await client.query(`
```

```

CREATE TABLE IF NOT EXISTS schema_migrations (
  version VARCHAR(50) PRIMARY KEY,
  applied_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
)
);
// 执行内嵌迁移
for (let i = 0; i < migrations.length; i++) {
  logger.info(` Migration ${i + 1}/${migrations.length}...`);
  await client.query(migrations[i]);
}
// 执行 migrations/ 目录中的 SQL 文件
const migrationsDir = path.resolve('./src/db/migrations');
if (fs.existsSync(migrationsDir)) {
  const files = fs.readdirSync(migrationsDir)
    .filter(f => f.endsWith('.sql'))
    .sort(); // 按文件名排序 (004, 005, 006...)
  for (const file of files) {
    const version = file.split('_')[0]; // 提取版本号 (如 "004")
    // 检查是否已执行
    const result = await client.query(
      `SELECT 1 FROM schema_migrations WHERE version = $1`,
      [version]
    );
    if (result.rows.length === 0) {
      logger.info(` SQL Migration ${file}...`);
      const sql = fs.readFileSync(`${migrationsDir}/${file}`, 'utf8');
      // 执行 SQL (可能包含多个语句)
      await client.query(sql);
      // 记录已执行
      await client.query(
        `INSERT INTO schema_migrations (version, applied_at) VALUES ($1, NOW()) ON CONFLICT DO NOTHING`,
        [version]
      );
      logger.info(` ✓ ${file} applied successfully`);
    } else {
      logger.info(` SQL Migration ${file} (skipped, already applied)`);
    }
  }
}
// 执行后迁移

```

```

logger.info('Running post-migrations (tsvector)...');
for (let i = 0; i < postMigrations.length; i++) {
  logger.info(' Post-migration ${i + 1}/${postMigrations.length}...');
  await client.query(postMigrations[i]);
}
logger.info('All migrations completed successfully.');
```

```

} finally {
  client.release();
}
}

// Run directly
if (process.argv[1] && process.argv[1].includes('migrate')) {
  migrate()
    .then(() => process.exit(0))
    .catch((err) => {
      logger.error('Migration failed:', { error: err.message });
      process.exit(1);
    });
}

export default migrate;
// ===== file: db/pool.js =====
import pg from 'pg';
import config from '../config.js';
import { logger } from '../utils/logger.js';
const { Pool } = pg;
const pool = new Pool({
  host: config.db.host,
  port: config.db.port,
  database: config.db.name,
  user: config.db.user,
  password: config.db.password,
  max: parseInt(process.env.DB_POOL_MAX) || 50, // 最大连接数（可配置，默认 50，P3 修复）
  idleTimeoutMillis: parseInt(process.env.DB_POOL_IDLE_TIMEOUT) || 30000, // 空闲连接超时（30秒）
  connectionTimeoutMillis: parseInt(process.env.DB_POOL_CONNECT_TIMEOUT) || 2000, // 连接超时（2秒）
  query_timeout: parseInt(process.env.DB_QUERY_TIMEOUT) || 30000, // 查询超时（30秒）
  keepAlive: true, // 保持连接活跃
  // 注意：statement_timeout 不在 Pool 配置中，移到 connect 事件中
});
// 连接级错误不应杀死进程（H4 修复）：短暂的 DB 抖动若触发全实例自杀会造成整体不可用。
// 改为只记录告警，依赖 pg Pool 自动重建连接；真正的持续故障交给编排器（k8s/docker）重启。
pool.on('error', (err) => {
  logger.error('[db] Unexpected error on idle client (auto-recovering):', {
    message: err.message,

```

```

[retentionDays]
);
return result.rowCount;
}
/**
 * 限制每个项的版本数量
 * @param {number} maxVersions - 每个项最大版本数（可选，默认使用配置值）
 * @returns {number} 清理的版本数量
 */
export async function limitVersionsPerItem(maxVersions = VERSION_CONFIG.maxVersionsPerItem) {
  const result = await pool.query(
    `DELETE FROM file_versions
    WHERE id IN (
    SELECT id FROM (
    SELECT id,
    ROW_NUMBER() OVER (PARTITION BY clipboard_item_id ORDER BY version_number DESC) as rn
    FROM file_versions
    ) t
    WHERE rn > $1
    )`,
    [maxVersions]
  );
  return result.rowCount;
}
/**
 * 获取版本统计信息
 * @param {string} userId - 用户ID
 * @returns {Object} 统计信息
 */
export async function getVersionStats(userId) {
  const result = await pool.query(
    `SELECT
    COUNT(*) as total_versions,
    COUNT(DISTINCT clipboard_item_id) as items_with_versions,
    COALESCE(SUM(content_size), 0) as total_size,
    COALESCE(AVG(content_size), 0) as avg_size,
    MIN(created_at) as oldest_version,
    MAX(created_at) as newest_version
    FROM file_versions
    WHERE user_id = $1`,
    [userId]
  );
  return result.rows[0];
}

```

```

/**
 * 启动版本自动清理调度器（应用启动时调用一次）。
 *
 * 背景：服务端已在 `PUT /api/clipboard/.id` 覆盖内容前自动落版本快照，
 * 从此每次内容变更都会往 file_versions 写一行。而 cleanupOldVersions /
 * limitVersionsPerItem 此前只挂在手动端点 `POST /api/versions/cleanup` 上，
 * VERSION_CONFIG.autoCleanupInterval 全仓库零引用 —— 表会无上限膨胀。
 * 本函数把这个配置真正用起来（默认每 1h 跑一次：超 90 天的、以及每条目超 50 版的）。
 *
 * 注意：测试环境不要调用，否则 setInterval 会挂住 jest 进程。
 */
export function startVersionCleanupScheduler() {
  const runCleanup = async () => {
    try {
      const byAge = await cleanupOldVersions();
      const byCount = await limitVersionsPerItem();
      if (byAge || byCount) {
        logger.info('[Versions] auto cleanup done', { byAge, byCount });
      }
    } catch (err) {
      // 清理是附属能力，失败绝不能影响主流程
      logger.error('[Versions] auto cleanup failed (non-fatal):', { error: err.message });
    }
  };
  // 启动时先跑一次，避免长期未重启的实例积攒历史脏数据
  runCleanup();
  const timer = setInterval(runCleanup, VERSION_CONFIG.autoCleanupInterval);
  // 不阻止进程退出
  if (typeof timer.unref === 'function') timer.unref();
  return timer;
}
export default {
  createVersion,
  getVersionHistory,
  getVersionDetail,
  restoreVersion,
  cleanupOldVersions,
  limitVersionsPerItem,
  getVersionStats,
  startVersionCleanupScheduler,
  VERSION_CONFIG,
};
// ===== file: utils/ws-redis-pubsub.js =====
/**
 * WebSocket Redis Pub/Sub 工具
 * 用于多实例部署时，通过 Redis 频道在实例间广播 WebSocket 消息
 *
 * 架构：

```

```

* - 每个实例保留本地 connections Map（存储实际 WebSocket 连接）
* - 当需要向用户广播消息时，发布到 Redis 频道 `ws:user:{userId}`
* - 所有实例订阅 `ws:user:*` 通配符，收到消息后转发给本地连接的设备
* - 实例 ID 用于避免自己发给自己（虽然无害，但浪费）
*/
import redis from 'redis';
import config from '../config.js';
import { logger } from './logger.js';
let subscriberClient = null; // Redis Pub/Sub 订阅客户端（独立连接）
let publisherClient = null; // Redis 发布客户端（复用主连接或独立）
let isRedisEnabled = false;
let instanceId = null; // 当前实例唯一 ID
let localConnections = null; // 引用本地 connections Map
/**
 * 初始化 WebSocket Redis Pub/Sub
 * @param {Map} connections - 本地 connections Map 的引用
 * @returns {Promise<boolean>} 是否成功初始化
 */
export async function initWsRedisPubSub(connections) {
  // 检查 Redis 是否配置
  if (!config.redis?.host) {
    logger.info('[WS Redis Pub/Sub] Redis not configured, using local broadcast only');
    return false;
  }
  try {
    localConnections = connections;
    instanceId = `instance:${process.pid}:${Date.now()}`;
    // 创建独立的订阅客户端（Redis Pub/Sub 需要专用连接）
    subscriberClient = redis.createClient({
      socket: {
        host: config.redis.host,
        port: config.redis.port,
      },
      password: config.redis.password || undefined,
      database: config.redis.database || 0,
    });
    subscriberClient.on('error', (err) => {
      logger.error('[WS Redis Pub/Sub] Subscriber error:', { error: err.message });
    });
    subscriberClient.on('connect', () => {
      logger.info('[WS Redis Pub/Sub] Subscriber connected');
    });
  }

```

```

await subscriberClient.connect();
// 订阅通配符频道：接收所有用户的消息
await subscriberClient.pSubscribe('ws:user:*', (message, channel) => {
  handleRedisMessage(message, channel);
});
isRedisEnabled = true;
logger.info('[WS Redis Pub/Sub] Initialized, instanceId=${instanceId}');
return true;
} catch (err) {
  logger.error('[WS Redis Pub/Sub] Failed to initialize:', { error: err.message });
  isRedisEnabled = false;
  return false;
}
}
/**
 * 处理从 Redis 收到的消息
 * @param {string} message - JSON 字符串消息
 * @param {string} channel - 频道名，格式：ws:user:{userId}
 */
function handleRedisMessage(message, channel) {
  try {
    const parsed = JSON.parse(message);
    const { userId, data, sourceInstanceId } = parsed;
    // 跳过自己发出的消息（避免重复发送）
    if (sourceInstanceId === instanceId) {
      return;
    }
    // 转发给本地连接的该用户设备
    const userDevices = localConnections.get(userId);
    if (!userDevices) {
      return;
    }
    for (const [deviceId, ws] of userDevices) {
      if (ws.readyState === 1) { // WebSocket.OPEN
        ws.send(JSON.stringify(data));
      }
    }
  } catch (err) {
    logger.error('[WS Redis Pub/Sub] Error handling message:', { error: err.message });
  }
}
/**
 * 向指定用户发布消息（通过 Redis Pub/Sub）

```

```

* @param {string} userId - 用户 ID
* @param {object} data - 要广播的消息对象
* @returns {Promise<void>}
*/
export async function publishToUser(userId, data) {
  if (!isRedisEnabled) {
    return; // Redis 未启用, 调用方会本地广播
  }
  try {
    // 复用主 Redis 客户端进行发布
    const { getRedisClient } = await import('./redis-client.js');
    const client = await getRedisClient();
    const message = {
      userId,
      data,
      sourceInstanceId: instanceId,
      timestamp: Date.now(),
    };
    const channel = `ws:user:${userId}`;
    await client.publish(channel, JSON.stringify(message));
  } catch (err) {
    logger.error('[WS Redis Pub/Sub] Failed to publish:', { error: err.message });
  }
}
/**
* 向所有在线用户发布系统通知 (通过 Redis Pub/Sub)
* @param {object} notification - 通知对象
* @returns {Promise<void>}
*/
export async function publishSystemNotification(notification) {
  if (!isRedisEnabled) {
    return;
  }
  try {
    const { getRedisClient } = await import('./redis-client.js');
    const client = await getRedisClient();
    const message = {
      type: 'system_notification',
      notification,
      sourceInstanceId: instanceId,
      timestamp: Date.now(),
    };
    // 发布到系统通知频道

```

```

await client.publish('ws:system', JSON.stringify(message));
} catch (err) {
  logger.error('[WS Redis Pub/Sub] Failed to publish system notification:', { error: err.message });
}
}
/**
 * 初始化系统通知订阅
 * @param {Function} onSystemNotification - 系统通知回调
 */
export async function initSystemNotificationSubscriber(onSystemNotification) {
  if (!subscriberClient) {
    return false;
  }
  try {
    await subscriberClient.subscribe('ws:system', (message) => {
      try {
        const parsed = JSON.parse(message);
        if (parsed.sourceInstanceId !== instanceId) {
          onSystemNotification(parsed.notification);
        }
      } catch (err) {
        logger.error('[WS Redis Pub/Sub] Error handling system notification:', { error: err.message });
      }
    });
    return true;
  } catch (err) {
    logger.error('[WS Redis Pub/Sub] Failed to subscribe to system notifications:', { error: err.message });
    return false;
  }
}
/**
 * 关闭 WebSocket Redis Pub/Sub 连接
 */
export async function closeWsRedisPubSub() {
  if (subscriberClient) {
    try {
      await subscriberClient.quit();
      subscriberClient = null;
      isRedisEnabled = false;
      logger.info('[WS Redis Pub/Sub] Closed');
    } catch (err) {
      logger.error('[WS Redis Pub/Sub] Error closing:', { error: err.message });
    }
  }
}
/**

```

```
* 检查 Redis Pub/Sub 是否启用
*/
export function isWsRedisEnabled() {
  return isRedisEnabled;
}
/**
* 获取当前实例 ID
*/
export function getInstanceId() {
  return instanceId;
}
// ===== file: validation/validator.js =====
/**
* 输入验证与防护模块
*
* 功能：
* 1. API参数验证
* 2. SQL注入防护
* 3. XSS防护
* 4. 通用数据清理
*/
/**
* 验证手机号格式
* @param {string} phone
* @returns {boolean}
*/
export function isValidPhone(phone) {
  // 中国大陆手机号：1开头，11位数字
  const phoneRegex = /^1[3-9]\d{9}$/;
  return phoneRegex.test(phone);
}
/**
* 验证验证码格式
* @param {string} code
* @returns {boolean}
*/
export function isValidCode(code) {
  // 6位数字验证码
  const codeRegex = /^[0-9]{6}$/;
  return codeRegex.test(code);
}
/**
* 验证UUID格式
* @param {string} uuid
* @returns {boolean}
```

```

*/
export function isValidUUID(uuid) {
const uuidRegex = /^[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}$/i;
return uuidRegex.test(uuid);
}
/**
* 验证内容类型
* @param {string} contentType
* @returns {boolean}
*/
export function isValidContentType(contentType) {
const validTypes = ['text', 'image', 'file', 'link', 'code'];
return validTypes.includes(contentType);
}
/**
* 验证设备类型
* @param {string} deviceType
* @returns {boolean}
*/
export function isValidDeviceType(deviceType) {
const validTypes = ['desktop', 'mobile', 'tablet', 'browser'];
return validTypes.includes(deviceType);
}
/**
* 验证平台类型
* @param {string} platform
* @returns {boolean}
*/
export function isValidPlatform(platform) {
const validPlatforms = ['windows', 'macos', 'linux', 'ios', 'android', 'browser'];
return validPlatforms.includes(platform);
}
/**
* 清理字符串输入（防XSS）
*
* 使用OWASP推荐的HTML实体转义防止XSS攻击
* 适用于HTML上下文的输出转义
*
* @param {string} input - 输入字符串
* @returns {string} - 转义后的安全字符串
*/
export function sanitizeString(input) {
if (typeof input !== 'string') return '';
// OWASP推荐的HTML实体转义
// 转义顺序很重要：必须先转义&，否则其他转义会被二次转义

```

```

return input
.replace(/&/g, '&amp;') // & 必须第一个转义
.replace(/</g, '&lt;') // < 标签开始
.replace(/>/g, '&gt;') // > 标签结束
.replace(/"/g, '&quot;') // " 属性引号
.replace(/'/g, '&#x27;') // ' 属性引号（十六进制）
.replace(/\\/g, '&#x2F;'); // \ 防止关闭标签
}
/**
 * 清理HTML内容（保留安全的HTML标签）
 *
 * 如果需要允许某些HTML标签（如<b>、<i>），使用此函数
 * 当前实现：移除所有HTML标签（保守做法）
 *
 * 注意：对于生产环境，建议使用专门的HTML清理库（如xss）
 *
 * @param {string} html - 输入HTML字符串
 * @returns {string} - 清理后的字符串
 */
export function sanitizeHtml(html) {
  if (typeof html !== 'string') return '';
  // 保守做法：移除所有HTML标签
  // 如需保留某些标签，应使用DOM解析+白名单
  return html.replace(/<[>]*>/g, '');
}
/**
 * 上下文感知的XSS防护
 *
 * 根据不同的输出上下文使用不同的转义策略
 */
/**
 * 用于HTML正文上下文的转义
 * 例：<div>此处是用户输入</div>
 */
export function escapeHtmlContext(input) {
  return sanitizeString(input);
}
/**
 * 用于HTML属性上下文的转义
 * 例：<div attr="此处是用户输入"></div>
 */
export function escapeAttributeContext(input) {
  if (typeof input !== 'string') return '';
  // 属性上下文需要转义引号和其他特殊字符

```

```

return input
.replace(/&/g, '&amp;')
.replace(/"/g, '&quot;')
.replace(/'/g, '&#x27;')
.replace(/</g, '&lt;')
.replace(/>/g, '&gt;')
.replace(/=/g, '&#x3D;')
.replace(/ /g, '&#x60;');
}
/**
 * 用于JavaScript字符串上下文的转义
 * 例：<script>var x = "此处是用户输入";</script>
 */
export function escapeJsContext(input) {
if (typeof input !== 'string') return '';
// JS字符串上下文：转义反斜杠、引号和换行
return input
.replace(/\\/g, '\\\\')
.replace(/"/g, '\\"')
.replace(/'/g, "'")
.replace(/\n/g, '\\n')
.replace(/\r/g, '\\r')
.replace(/ /g, '\\ ');
}
/**
 * 用于URL上下文的转义
 * 例：<a href="此处是用户输入">
 */
export function escapeUrlContext(input) {
if (typeof input !== 'string') return '';
// URL编码：使用encodeURIComponent
try {
return encodeURIComponent(input);
} catch (err) {
return '';
}
}
/**
 * 防SQL注入：转义特殊字符
 * 注意：使用参数化查询时不需要这个
 * @param {string} input
 * @returns {string}
 */
export function escapeSql(input) {
if (typeof input !== 'string') return '';

```

```

return input
.replace(/'/g, "'")
.replace(/\//g, '\\')
.replace(/\0/g, '\\0');
}
/**
 * 验证并清理分页参数
 * @param {any} page
 * @param {any} limit
 * @returns {{page: number, limit: number}}
 */
export function validatePagination(page, limit, opts = {}) {
const { all } = opts
if (all) return { page: 1, limit: Infinity }
let pageNum = parseInt(page) || 1;
let limitNum = parseInt(limit) || 50;
// 限制范围
pageNum = Math.max(1, Math.min(1000, pageNum));
limitNum = Math.max(1, Math.min(100, limitNum));
return { page: pageNum, limit: limitNum };
}
/**
 * 验证并清理搜索参数
 * @param {string} search
 * @param {number} maxLength
 * @returns {string}
 */
export function validateSearch(search, maxLength = 100) {
if (typeof search !== 'string') return "";
// 清理并限制长度
return sanitizeString(search.trim()).substring(0, maxLength);
}
/**
 * 验证JSON数据
 * @param {any} data
 * @returns {boolean}
 */
export function isValidJson(data) {
try {
if (typeof data === 'string') {
JSON.parse(data);
}
}
return true;
}

```

```
} catch (e) {
return false;
}
}
/**
 * 验证URL格式
 * @param {string} url
 * @returns {boolean}
 */
export function isValidUrl(url) {
try {
new URL(url);
return true;
} catch (e) {
return false;
}
}
/**
 * 验证邮箱格式
 * @param {string} email
 * @returns {boolean}
 */
export function isValidEmail(email) {
const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
return emailRegex.test(email);
}
/**
 * 验证昵称
 * @param {string} nickname
 * @returns {{valid: boolean, error?: string}}
 */
export function validateNickname(nickname) {
if (!nickname || typeof nickname !== 'string') {
return { valid: false, error: 'Nickname cannot be empty' };
}
const trimmed = nickname.trim();
if (trimmed.length === 0) {
return { valid: false, error: 'Nickname cannot be empty' };
}
if (trimmed.length > 50) {
return { valid: false, error: 'Nickname cannot exceed 50 characters' };
}
}
// 检查是否包含特殊字符
```

```

const invalidChars = /[<>'&]/;
if (invalidChars.test(trimmed)) {
  return { valid: false, error: 'Nickname contains invalid characters' };
}
return { valid: true };
}
/**
 * 验证设备名称
 * @param {string} name
 * @returns {{valid: boolean, error?: string}}
 */
export function validateDeviceName(name) {
  if (!name || typeof name !== 'string') {
    return { valid: false, error: 'Device name cannot be empty' };
  }
  const trimmed = name.trim();
  if (trimmed.length === 0) {
    return { valid: false, error: 'Device name cannot be empty' };
  }
  if (trimmed.length > 100) {
    return { valid: false, error: 'Device name cannot exceed 100 characters' };
  }
  return { valid: true };
}
/**
 * 验证剪贴板内容
 * @param {object} data
 * @returns {{valid: boolean, error?: string}}
 */
export function validateClipboardData(data) {
  const { sourceDeviceId, contentType, contentEncrypted } = data;
  if (!sourceDeviceId) {
    return { valid: false, error: 'sourceDeviceId is required' };
  }
  if (!isValidUUID(sourceDeviceId)) {
    return { valid: false, error: 'Invalid sourceDeviceId format' };
  }
  if (!contentType) {
    return { valid: false, error: 'contentType is required' };
  }
}

```

```
if (!isValidContentType(contentType)) {
  return { valid: false, error: 'Invalid contentType' };
}
if (!contentEncrypted) {
  return { valid: false, error: 'contentEncrypted is required' };
}
if (typeof contentEncrypted !== 'string') {
  return { valid: false, error: 'contentEncrypted must be a string' };
}
if (contentEncrypted.length > 10 * 1024 * 1024) { // 10MB limit
  return { valid: false, error: 'Content too large' };
}
return { valid: true };
}
/**
 * 验证设备注册数据
 * @param {object} data
 * @returns {{valid: boolean, error?: string}}
 */
export function validateDeviceData(data) {
  const { deviceName, deviceType, platform } = data;
  const nameValidation = validateDeviceName(deviceName);
  if (!nameValidation.valid) {
    return nameValidation;
  }
  if (!deviceType) {
    return { valid: false, error: 'deviceType is required' };
  }
  if (!isValidDeviceType(deviceType)) {
    return { valid: false, error: 'Invalid deviceType' };
  }
  if (!platform) {
    return { valid: false, error: 'platform is required' };
  }
  if (!isValidPlatform(platform)) {
    return { valid: false, error: 'Invalid platform' };
  }
  return { valid: true };
}
```

```
}
/**
 * 验证中间件工厂
 * @param {Function} validator
 * @param {Function} extractor
 * @returns {Function}
 */
export function validationMiddleware(validator, extractor = (req) => req.body) {
  return (req, res, next) => {
    const data = extractor(req);
    const result = validator(data);
    if (!result.valid) {
      return res.status(400).json({ error: result.error });
    }
    next();
  };
}
/**
 * 常用验证中间件
 */
export const validators = {
  // 验证登录数据
  login: validationMiddleware((data) => {
    const { phone, code } = data;
    if (!phone) {
      return { valid: false, error: 'Phone number is required' };
    }
    if (!isValidPhone(phone)) {
      return { valid: false, error: 'Invalid phone number format' };
    }
    if (!code) {
      return { valid: false, error: 'Verification code is required' };
    }
    if (!isValidCode(code)) {
      return { valid: false, error: 'Invalid verification code format' };
    }
    return { valid: true };
  }),
  // 验证发送验证码
  sendCode: validationMiddleware((data) => {
```

```
const { phone } = data;
if (!phone) {
  return { valid: false, error: 'Phone number is required' };
}
if (!isValidPhone(phone)) {
  return { valid: false, error: 'Invalid phone number format' };
}
return { valid: true };
}),
// 验证设备注册
registerDevice: validationMiddleware(validateDeviceData),
// 验证剪贴板数据
createClipboard: validationMiddleware(validateClipboardData),
};
export default {
  isValidPhone,
  isValidCode,
  isValidUUID,
  isValidContentType,
  isValidDeviceType,
  isValidPlatform,
  sanitizeString,
  sanitizeHtml,
  escapeSql,
  validatePagination,
  validateSearch,
  isValidJson,
  isValidUrl,
  isValidEmail,
  validateNickname,
  validateDeviceName,
  validateClipboardData,
  validateDeviceData,
  validationMiddleware,
  validators,
};
// ===== file: ws/server.js =====
import { WebSocketServer } from 'ws';
import jwt from 'jsonwebtoken';
import pool from '../db/pool.js';
import config from '../config.js';
import { logger } from '../utils/logger.js';
import { getRedisClient, checkWsConnectionLimit, removeWsConnection } from '../middleware/rateLimiter.js';
```

```

import { setWsConnections } from './middleware/metrics.js';
import { isMaintenanceOn } from './middleware/maintenance.js';
/**
 * AF-41：按设备强制断开（管理平台「远程下线」消费）。
 * 找到该用户名下目标设备的活跃 WS 连接 → 推送 force_logout 指令 → 以 4003 关闭。
 * 客户端收到 force_logout 后清除本地登录态并回登录页；连接表由 close 处理器统一摘除。
 * @returns {boolean} 是否找到并断开了活跃连接
 */
export function forceDisconnectDevice(userId, deviceId, reason) {
  const ws = connections.get(userId)?.get(deviceId);
  if (!ws) return false;
  try {
    ws.send(JSON.stringify({ type: 'force_logout', reason: reason || '' }));
  } catch {
    /* 发送失败不阻塞关闭 */
  }
  try {
    ws.close(4003, 'Force offline by admin');
  } catch {
    /* ignore */
  }
  logger.info('Force disconnect requested for device ${deviceId} (user ${userId})');
  return true;
}
import { createNotification } from './services/notificationService.js';
import {
  initWsRedisPubSub,
  publishToUser,
  closeWsRedisPubSub,
  isWsRedisEnabled,
} from './utils/ws-redis-pubsub.js';
// Map<userId, Map<deviceId, WebSocket>>
const connections = new Map();
// CO-02: 当前实例活跃 WS 连接数（含尚未注册/被拒的连接），经 setWsConnections 上报 Prometheus gauge
let liveWsConnections = 0;
// 防止同一 server 实例被多次调用（测试环境会导致内存泄漏）
const _setupServers = new WeakMap();
// 未注册连接的超时时间（毫秒）
const UNREGISTERED_TIMEOUT = 10000; // 10秒
// WebSocket消息速率限制
const WS_RATE_LIMIT = 50; // 每秒最多50条消息
const WS_RATE_WINDOW = 1000; // 1秒窗口
// 单连接发送缓冲上限（P5 背压）：超过则丢弃该消息，避免慢客户端导致服务端内存无限增长

```

```
const WS_MAX_BUFFERED = 1024 * 1024; // 1MB
// 带背压保护的发送：慢客户端（bufferedAmount 过高）直接丢弃，保护服务端内存
function safeSend(ws, payload) {
  if (ws.readyState !== 1) return false; // 仅 OPEN 状态
  if (typeof ws.bufferedAmount === 'number' && ws.bufferedAmount > WS_MAX_BUFFERED) {
    logger.warn('[WebSocket] backpressure: dropping message, bufferedAmount too high', {
      bufferedAmount: ws.bufferedAmount,
    });
    return false;
  }
  try {
    ws.send(payload);
    return true;
  } catch (err) {
    logger.error('[WebSocket] send error:', { error: err.message });
    return false;
  }
}

export function setupWebSocket(server) {
  // 幂等性检查：同一个 server 只创建一次 WebSocket 服务
  if (_setupServers.has(server)) {
    return _setupServers.get(server);
  }
  // WebSocket 服务（限制消息大小为 1MB，防止内存耗尽）
  const wss = new WebSocketServer({
    server,
    path: '/ws',
    maxPayload: 1024 * 1024 // 1MB
  });
  _setupServers.set(server, wss);
  // 测试环境大幅提升 listener 上限，避免 MaxListenersExceededWarning
  const maxListeners = process.env.NODE_ENV === 'test' ? 500 : 100;
  wss.setMaxListeners(maxListeners);
  if (server.setMaxListeners) server.setMaxListeners(maxListeners);
  // 初始化 WebSocket Redis Pub/Sub（如果 Redis 已配置）
  initWsRedisPubSub(connections).then((enabled) => {
    if (enabled) {
      logger.info('[WebSocket] Redis Pub/Sub enabled for multi-instance deployment');
    } else {
      logger.info('[WebSocket] Redis Pub/Sub disabled, using local broadcast only');
    }
  });
  wss.on('connection', async (ws, req) => {
    // CO-02: 连接建立即计数，close 时递减；close 监听必须最先挂载，
```

```

// 否则 origin/token/CSRF 校验被提前 return 拒绝的连接只增不减导致 gauge 泄漏
liveWsConnections++;
setWsConnections(liveWsConnections);
ws.on('close', () => {
  liveWsConnections = Math.max(0, liveWsConnections - 1);
  setWsConnections(liveWsConnections);
});
logger.info('WebSocket connection attempt');
// ===== CO-20: 维护模式拦截（仅握手阶段查一次，带 5s 缓存，不逐消息检查） =====
// upgrade 不经 Express 中间件链，故在 connection 入口拒绝；isMaintenanceOn 读库失败内部兜底为放行
if (await isMaintenanceOn()) {
  logger.warn('WebSocket connection rejected: Maintenance mode');
  ws.close(4003, 'Maintenance mode');
  return;
}
// ===== 安全增强 #1: Origin 验证 =====
const origin = req.headers.origin;
if (config.nodeEnv === 'production' && origin) {
  const allowedOrigins = config.cors?.origins || [];
  if (!allowedOrigins.includes(origin)) {
    logger.warn('WebSocket connection rejected: Invalid origin', { origin });
    ws.close(4003, 'Invalid origin');
    return;
  }
}
// ===== 安全增强 #2: 解析并验证 Token =====
const url = new URL(req.url, `http://${req.headers.host}`);
const token = url.searchParams.get('token') || req.headers['authorization']?.split(' ')[1];
if (!token) {
  ws.close(4001, 'Authentication required');
  return;
}
let decoded;
try {
  decoded = jwt.verify(token, config.jwt.secret);
} catch (err) {
  ws.close(4002, 'Invalid token');
  return;
}
// Redis 客户端必须在 CSRF 校验之前获取：
// 此前声明位于下方黑名单检查处，CSRF 块引用时触发 TDZ ReferenceError，
// 导致任何携带 csrf_token 的连接（含生产环境全部连接）fail-closed 被拒。
const redis = await getRedisClient();

```

```
// ===== 安全增强 #2.5: WebSocket CSRF 保护 =====
const csrfToken = url.searchParams.get('csrf_token');
if (config.nodeEnv === 'production' || csrfToken) {
  // 生产环境必须验证CSRF token
  if (!csrfToken) {
    logger.warn('WebSocket connection rejected: Missing CSRF token', { userId: decoded.userId });
    ws.close(4006, 'CSRF token required');
    return;
  }
  // 格式检查
  if (!/^[a-f0-9]{64}$/i.test(csrfToken)) {
    logger.warn('WebSocket connection rejected: Invalid CSRF token format', { userId: decoded.userId });
    ws.close(4006, 'Invalid CSRF token');
    return;
  }
  // Redis验证：检查token是否存在且属于该用户
  if (redis) {
    try {
      const tokenData = await redis.get(`csrf:${csrfToken}`);
      if (!tokenData) {
        logger.warn('WebSocket connection rejected: CSRF token not found in Redis', { userId: decoded.userId });
        ws.close(4006, 'Invalid CSRF token');
        return;
      }
      const parsed = JSON.parse(tokenData);
      if (parsed.userId !== decoded.userId) {
        logger.warn('WebSocket connection rejected: CSRF token user mismatch', {
          userId: decoded.userId,
          tokenUserId: parsed.userId
        });
        ws.close(4006, 'Invalid CSRF token');
        return;
      }
    } catch (err) {
      logger.error('CSRF token validation error', { error: err.message });
    }
  }
  // 验证通过，使用后删除token（单次使用）
  await redis.del(`csrf:${csrfToken}`);
} else {
  // Redis不可用时，如果是最佳环境则拒绝连接
  if (config.nodeEnv === 'production') {
    logger.error('WebSocket CSRF validation failed: Redis unavailable in production');
  }
}
```

```
ws.close(4006, 'Service temporarily unavailable');
return;
}
}
}
// ===== 安全增强 #3: 检查 Token 是否在黑名单中 =====
if (redis && decoded.jti) {
  const blacklisted = await redis.get('bl:${decoded.jti}');
  if (blacklisted) {
    logger.warn('WebSocket connection rejected: Token revoked', { userId: decoded.userId });
    ws.close(4004, 'Token revoked');
    return;
  }
}
const userId = decoded.userId;
let deviceId = null;
let isRegistered = false;
let messageCount = 0;
let lastMessageTime = Date.now();
// ===== 安全增强 #4: 未注册连接超时 =====
const unregisteredTimeout = setTimeout(() => {
  if (!isRegistered) {
    logger.warn('WebSocket connection timeout: Not registered', { userId });
    ws.close(4005, 'Registration timeout');
  }
}, UNREGISTERED_TIMEOUT);
// ===== 安全增强 #5: 消息速率限制 =====
function checkRateLimit() {
  const now = Date.now();
  if (now - lastMessageTime > WS_RATE_WINDOW) {
    // 新窗口
    messageCount = 1;
    lastMessageTime = now;
    return true;
  }
  if (messageCount >= WS_RATE_LIMIT) {
    logger.warn('WebSocket rate limit exceeded', { userId, deviceId });
    return false;
  }
  messageCount++;
  return true;
}
ws.on('message', async (data) => {
```

```

// 速率限制检查
if (!checkRateLimit()) {
  ws.send(JSON.stringify({ type: 'error', message: 'Rate limit exceeded' }));
  return;
}
try {
  const message = JSON.parse(data.toString());
  switch (message.type) {
    case 'register':
      deviceId = message.deviceId;
      if (!deviceId) {
        ws.send(JSON.stringify({ type: 'error', message: 'deviceId required' }));
        return;
      }
      // Verify device belongs to user
      const deviceCheck = await pool.query(
        'SELECT id FROM devices WHERE id = $1 AND user_id = $2',
        [deviceId, userId]
      );
      if (deviceCheck.rows.length === 0) {
        ws.send(JSON.stringify({ type: 'error', message: 'Device not found' }));
        return;
      }
      // 连接数限制 (H6 修复：此前 checkWsConnectionLimit 定义但未调用，限制形同虚设)
      if (!checkWsConnectionLimit(userId, deviceId)) {
        logger.warn('[WebSocket] connection limit exceeded', { userId, deviceId });
        ws.send(JSON.stringify({ type: 'error', message: 'Connection limit exceeded (max 5 per user)' }));
        ws.close(4007, 'Connection limit exceeded');
        return;
      }
      // Store connection
      if (!connections.has(userId)) {
        connections.set(userId, new Map());
      }
      // 重连场景：同一 deviceId 的旧连接若仍挂在广播表中，先移除并关闭，
      // 避免重连风暴期间旧连接未及时清理而累积到每用户 5 连接上限被误杀（4007）
      const userConns = connections.get(userId);
      const oldWs = userConns.get(deviceId);
      if (oldWs && oldWs !== ws) {
        userConns.delete(deviceId);
        try { oldWs.close(4000, 'Replaced by new connection'); } catch { /* ignore */ }
      }

```

```
connections.get(userId).set(deviceId, ws);
// 标记已注册并清除超时计时器
isRegistered = true;
clearTimeout(unregisteredTimeout);
// Update device online status
await pool.query(
  'UPDATE devices SET is_online = TRUE, last_seen_at = NOW() WHERE id = $1',
  [deviceId]
);
ws.send(JSON.stringify({ type: 'registered', deviceId }));
logger.info('Device ${deviceId} registered for user ${userId}');
break;
case 'clipboard':
  // Broadcast clipboard content to other devices of the same user
  if (!deviceId) {
    ws.send(JSON.stringify({ type: 'error', message: 'Register device first' }));
    return;
  }
  const clipboardMessage = {
    type: 'clipboard',
    sourceDeviceId: deviceId,
    content: message.content,
    contentType: message.contentType,
    timestamp: Date.now(),
  };
  // 本地广播（同一实例内的其他设备）
  const userDevices = connections.get(userId);
  if (userDevices) {
    for (const [otherDeviceId, otherWs] of userDevices) {
      if (otherDeviceId !== deviceId) {
        safeSend(otherWs, JSON.stringify(clipboardMessage));
      }
    }
  }
  // Redis Pub/Sub 广播（其他实例的设备）
  if (isWsRedisEnabled()) {
    await publishToUser(userId, clipboardMessage);
  }
  break;
case 'ping':
  ws.send(JSON.stringify({ type: 'pong', timestamp: Date.now() }));
```

```
// Update heartbeat
if (deviceId) {
  await pool.query(
    'UPDATE devices SET last_seen_at = NOW() WHERE id = $1',
    [deviceId]
  );
}
break;
default:
  ws.send(JSON.stringify({ type: 'error', message: 'Unknown message type: ${message.type}' }));
}
} catch (err) {
  logger.error('WebSocket message error:', { error: err.message });
  ws.send(JSON.stringify({ type: 'error', message: 'Invalid message format' }));
}
});
ws.on('close', async () => {
  // 清除未注册超时计时器
  clearTimeout(unregisteredTimeout);
  if (deviceId) {
    // Remove from connection-limit tracker (H6 修复)
    removeWsConnection(userId, deviceId);
    // Remove from connections
    const userDevices = connections.get(userId);
    if (userDevices) {
      userDevices.delete(deviceId);
      if (userDevices.size === 0) {
        connections.delete(userId);
      }
    }
  }
  // Update device offline status
  try {
    await pool.query(
      'UPDATE devices SET is_online = FALSE WHERE id = $1',
      [deviceId]
    );
  } catch (err) {
    logger.error('Failed to update device offline status:', { error: err.message });
  }
  logger.info('Device ${deviceId} disconnected');
}
});
ws.on('error', (err) => {
```

```
logger.error('WebSocket error:', { error: err.message });
});
// Start heartbeat check
startHeartbeatCheck(ws, deviceId);
});
// Heartbeat check: close connections that haven't sent ping
function startHeartbeatCheck(ws, deviceId) {
  const interval = setInterval(() => {
    if (ws.readyState !== 1) {
      clearInterval(interval);
      return;
    }
    ws.isAlive = false;
    ws.ping();
    const timeout = setTimeout(() => {
      if (!ws.isAlive) {
        ws.terminate();
      }
    }, config.ws.heartbeatTimeout);
    ws.once('pong', () => {
      ws.isAlive = true;
      clearTimeout(timeout);
    });
  }, config.ws.heartbeatInterval);
  ws.on('close', () => clearInterval(interval));
}
return wss;
}
// Broadcast to all devices of a user
export function broadcastToUser(userId, message) {
  // 本地广播（当前实例内的设备）
  const userDevices = connections.get(userId);
  if (userDevices) {
    for (const [deviceId, ws] of userDevices) {
      safeSend(ws, JSON.stringify(message));
    }
  }
  // Redis Pub/Sub 广播（其他实例的设备）
  if (isWsRedisEnabled()) {
    publishToUser(userId, message).catch((err) => {
      logger.error('[WebSocket] Redis Pub/Sub broadcast failed:', { error: err.message });
    });
  }
}
```

```

});
}
}
// Broadcast to every connected client on this instance (多实例部署时配合客户端轮询兜底)。
// 用于全局配置变更广播, 如 feature_flags.updated——开关类消息允许最终一致 (客户端 5s TTL 兜底)。
export function broadcastToAllClients(message) {
  let count = 0;
  const payload = JSON.stringify(message);
  for (const [, devices] of connections) {
    for (const [, ws] of devices) {
      if (ws.readyState === 1) {
        try {
          ws.send(payload);
          count++;
        } catch {
          // 忽略发送失败
        }
      }
    }
  }
  logger.info('[WebSocket] broadcastToAllClients: ${message?.type || 'unknown'} → ${count} clients');
  return count;
}
// Send notification to all devices of a user
// 同时持久化到 notification_history (此前只推 WS 不落库, 导致 GET /history 永远为空)
export async function sendNotification(userId, notification) {
  const notificationType = notification.notificationType || 'sync_complete';
  // 1) 落库 (UPSERT 风格: createNotification 已处理写入)
  let persistedId = null;
  try {
    const saved = await createNotification({
      userId,
      notificationType,
      title: notification.title,
      content: notification.body,
      metadata: notification.data || {},
    });
    persistedId = saved?.id ?? null;
  } catch (err) {
    logger.error('[WS] 通知落库失败 (仍继续推送): ', { error: err?.message, userId });
  }
  // 2) 推送 WS (携带真实 id + type, 供前端映射分类与去重)
  const message = {
    type: 'notification',
    id: persistedId != null ? String(persistedId) : `notif_${Date.now()}`,
    notificationType,

```

```

title: notification.title,
body: notification.body,
data: notification.data || {},
timestamp: new Date().toISOString(),
};
broadcastToUser(userId, message);
logger.info('Notification sent', {
  userId,
  notificationType,
  title: notification.title,
  onlineDevices: getOnlineDeviceCount(userId),
});
}
}
/**
 * 登录时检测是否为「新设备 / 新登录地点」登录；若是则推送 security_alert。
 * 必须在 createSessionAndGenerateToken 之前调用（此时本次会话尚未入库，历史会话数不含本次）。
 * @param {string} userId
 * @param {{ deviceName?: string, platform?: string, ip?: string }} ctx
 */
export async function detectAndNotifyNewLogin(userId, ctx = {}) {
  const deviceName = (ctx.deviceName || 'Unknown Device').toString();
  const platform = (ctx.platform || 'unknown').toString();
  const ip = ctx.ip || '';
  try {
    const prior = await pool.query('SELECT COUNT(*)::bigint AS cnt FROM user_sessions WHERE user_id = $1', [userId]);
    const priorCount = Number(prior.rows[0]?.cnt || 0);
    if (priorCount === 0) return; // 首次登录（新注册）不报警
    const seen = await pool.query(
      'SELECT 1 FROM user_sessions WHERE user_id = $1 AND device_name = $2 AND platform = $3 LIMIT 1',
      [userId, deviceName, platform]
    );
    if (seen.rows.length > 0) return; // 已知设备，不报警
    await sendNotification(userId, {
      notificationType: 'security_alert',
      title: 'New login detected',
      body: `A new device (${deviceName} · ${platform})${ip ? ` from ${ip}` : ''} just signed in to your account.`,
      data: { deviceName, platform, ip },
    });
  } catch (err) {
    logger.error('[WS] 新登录安全警报检测失败（已忽略）:', { error: err?.message, userId });
  }
}
// Get online device count for a user
export function getOnlineDeviceCount(userId) {
  const userDevices = connections.get(userId);
  return userDevices ? userDevices.size : 0;
}

```

```

}
// Check if a user has any online devices
export function hasOnlineDevices(userId) {
  return getOnlineDeviceCount(userId) > 0;
}
// Get all online users (for admin/monitoring)
export function getOnlineUsers() {
  const onlineUsers = [];
  for (const [userId, devices] of connections) {
    if (devices.size > 0) {
      onlineUsers.push({
        userId,
        deviceCount: devices.size,
        devices: Array.from(devices.keys()),
      });
    }
  }
  return onlineUsers;
}
export { connections };
/**
 * 优雅关闭 WebSocket 服务器
 * 1. 通知所有连接的客户端准备重启（发送 reconnect 消息）
 * 2. 等待客户端断开连接（设置超时）
 * 3. 返回 Promise，关闭完成后 resolve
 * @param {number} timeout - 等待客户端断开的超时时间（毫秒），默认 10 秒
 * @returns {Promise<void>}
 */
export async function gracefulShutdown(timeout = 10000) {
  logger.info('[WebSocket] Starting graceful shutdown...');
  // 1. 通知所有客户端重连
  const reconnectMessage = JSON.stringify({
    type: 'server_shutdown',
    message: 'Server is restarting, please reconnect in 5 seconds',
    reconnectAfter: 5000,
    timestamp: Date.now(),
  });
  let clientCount = 0;
  for (const [userId, devices] of connections) {
    for (const [deviceId, ws] of devices) {
      if (ws.readyState === 1) { // WebSocket.OPEN
        try {
          ws.send(reconnectMessage);
          clientCount++;
        }
      }
    }
  }
}

```

```

} catch (err) {
// 忽略发送失败
}
}
}
}
logger.info('[WebSocket] Notified ${clientCount} clients to reconnect');
// 2. 等待客户端断开 (设置超时)
return new Promise((resolve) => {
const startTime = Date.now();
const checkInterval = setInterval(() => {
let remainingClients = 0;
for (const [userId, devices] of connections) {
for (const [deviceId, ws] of devices) {
if (ws.readyState === 1) { // WebSocket.OPEN
remainingClients++;
}
}
}
if (remainingClients === 0) {
clearInterval(checkInterval);
logger.info('[WebSocket] All clients disconnected, shutdown complete');
resolve();
} else if (Date.now() - startTime > timeout) {
clearInterval(checkInterval);
logger.warn('[WebSocket] Graceful shutdown timeout, ${remainingClients} clients still connected');
resolve(); // 超时后强制退出
}
}, 500);
// 3. 同时设置超时保护
setTimeout(() => {
clearInterval(checkInterval);
logger.warn('[WebSocket] Graceful shutdown forced timeout');
resolve();
}, timeout);
});
}
// 进程退出时清理 Redis Pub/Sub 连接
process.on('exit', () => {
closeWsRedisPubSub();
});
process.on('SIGINT', () => {
closeWsRedisPubSub();
});

```

```
process.exit(0);  
});
```